



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance measure
E – Environment
A – Actuators
S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.food_positions
self.opponents
self.agent_pos
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline environment is a Multi-Agent Environment because it includes `self.opponents`, which represents other agents present in the grid. These opponents move independently and can interact with the main agent by causing collisions and affecting its score. So multiple agents influence the environment making it a multi-

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete cumulative history of all percepts received by an agent from its initialization up to the current moment. It contains all the information the agent has perceived from the environment over time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) component of the PEAS framework because it gathers information about the environment and returns the percepts that the agent can observe.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Fully Observable because the agent receives all relevant information about its current state and the environment through `get_percept()`, including its position, opponent positions, collision status, score, and food-related information. Since no important state information is hidden from the agent, it can be

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When `execute_action()` deducts points for hitting a wall, the Performance Measure (P) component is being updated because the agent's score is changed to reflect poorer performance

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

real effects of the agent's actions on the environment, not its internal processing effort. Penalizing wall collisions measures an actual negative outcome in the environment and encourages better decision-making. If the metric rewarded internal processing, instead, the agent could exploit the metric by performing unnecessary

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

By adding `self.toxic_traps` but hiding them from the agent's sensors, the environment changes from fully observable to partially observable. This is because the agent cannot directly observe the locations of the traps, meaning some relevant

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the 'smells_toxin' sensor expands the agent's percept sequence by providing information about toxic traps. This allows the agent to recognize dangerous locations and avoid actions that would reduce its score. So the sensor helps the agent act more rationally by maximizing expected utility based on the information available to

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

every time it collected dirt. To maximize its score, the agent could clean a dirt patch, dump the dirt back onto the floor, and clean it again repeatedly. This allowed the agent to earn infinite points without keeping the environment clean, demonstrating the danger of poorly designed performance metrics.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING